

API INTEGRATION IN THIS APP

The generic axios + React Query pattern, and what this repo does instead — a companion to the "API Integration in React" reference deck

1. Same-origin proxy → 2. axios.ts → 3. One hook = one operation → 4. Inline types → 5. Pages compose hooks

i Why this looks different from the reference deck

The reference deck teaches a **production-scale** pattern: React Query for caching, a `services/` layer, a central `types/` folder, a central `API_ENDPOINTS` registry. This repo is a **teaching demo** of the raw request flow (see `CLAUDE.md`) — it deliberately skips those layers so the HTTP mechanics stay visible in one place per hook. Neither is "wrong" — they're solving different problems.

1 Generic pattern vs. this repo, side by side

CONCERN	REFERENCE DECK (GENERIC)	THIS REPO
API base URL	VITE_API_URL env var + CORS + withCredentials	Vite dev-server proxy — same-origin, no CORS setup needed at all
Caching / loading state	React Query (useQuery/useMutation)	Manual isLoading/isError/error state per hook
Endpoint paths	Central API_ENDPOINTS registry object	Each hook inlines its own path string
Request/response types	Central types/ folder, shared	Duplicated inline in each hook file, on purpose
Service layer	services/ — pure functions, no React	Merged into the hook itself — one file, one operation
Access token storage	In-memory JS variable	httpOnly cookie — page JS never touches either token (see the JWT doc)

2 This repo's real folder structure

```

client/src/
├── lib/axios.ts — shared instance + refresh interceptor
├── features/consumers/
│   ├── get/useConsumers.ts, useConsumerDetail.ts
│   ├── post/useCreateConsumer.ts
│   ├── patch/useUpdateConsumer.ts · one hook per operation, not per resource
│   ├── put/useReplaceConsumer.ts
│   └── delete/useDeleteConsumer.ts
├── features/auth/useAuth.tsx — session Context, polls /auth/me
├── components/ — login-page, signup-page, dashboard-header, ui/ (shadcn)
├── pages/ — ConsumersListPage, ConsumerDetailPage, DashboardHome...
└── App.tsx — React Router routes (ported from Next.js)
  
```

1 The Vite proxy replaces VITE_API_URL entirely

client/vite.config.ts

```
server: {
  port: 5173,
  proxy: {
    "/api": { target: "http://localhost:8787", changeOrigin: true },
  },
},
```

- Every `fetch("/api/...")` or `api.get("/...")` the browser makes looks same-origin — Vite silently forwards it to Express on :8787.
- Because it's same-origin, the `httpOnly access_token/refresh_token` cookies are sent automatically. No `VITE_API_URL`, no `withCredentials: true`, no CORS headers on the server at all.
- Trade-off: this only works because client and server are dev-served together. A real deployment splitting them onto different domains would need the reference deck's env-var + CORS approach instead.

2 One shared axios instance, not a bare client + separate config

client/src/lib/axios.ts

```
export const api = axios.create({
  baseURL: "/api",
  headers: { "Content-Type": "application/json" },
});
```

- Plays the same role as the reference deck's `apiClient` — but no `Authorization: Bearer` header logic, because there's no in-memory token to attach. The cookie does that job invisibly.
- Its response interceptor (`401` → refresh → retry) is the whole subject of `AXIOS-INTERCEPTORS-EXPLAINED.pdf` — not repeated here.

Why no API_ENDPOINTS registry? With 6 hooks total and one resource (consumers), a central registry would be one extra file to open per request instead of reading the path inline where it's used. The reference deck's registry pays off once you have dozens of endpoints across many resources — this repo intentionally isn't at that scale.

3 The hook *is* the service — by design

client/src/features/consumers/get/useConsumers.ts (shape, not full code)

```
interface Consumer { id: string; firstName: string; /* ... */ } // ← types live HERE

export function useConsumers({ search, page, pageSize }) {
  const [consumers, setConsumers] = useState([]);
  const [isLoading, setIsLoading] = useState(true); // ← no useQuery
  const [isError, setIsError] = useState(false);

  useEffect(() => {
    /* api.get("/consumers", { params }) then setConsumers(...) */
  }, [search, page, pageSize]);

  return { consumers, isLoading, isError }; // ← component reads this directly
}
```

- No `productService.list()` indirection — the network call and the React state that tracks it live in the same file, so you can read top-to-bottom what a component's data actually does.
- The `Consumer` interface is redefined in every consumer hook rather than imported from a shared `types/` file — CLAUDE.md calls this out explicitly as intentional duplication, not an oversight.

4 No cache, no `invalidateQueries` — a manual refetch instead

- The reference deck's `onSuccess: () => qc.invalidateQueries(...)` pattern needs a query-key cache to invalidate. This repo has none — after a mutation, the calling component re-triggers its own `useEffect` (e.g. by re-navigating or re-running `load()`) to see fresh data.
- Cost: no automatic background refetch, no shared cache between components asking for the same data twice. Benefit: there is no cache invalidation bug to explain — a real category of React Query confusion for beginners.

Two files already show the road not taken: `client/src/lib/fetch-client.ts` sketches a centralized `res.ok` check, and `lib/simple-cache.ts` sketches a hand-rolled React-Query-style cache. Both are kept unused on purpose, as reference — read them to see the abstraction this repo chose not to take, without it changing any real behavior.

5 Components stay thin — same end goal as the reference deck



- This is the exact same shape as the reference deck's step 9 (ProductsPage) — "always handle all four states." The difference is only *where* those four states come from: a React Query result object there, a hand-rolled `{consumers, isLoading, isError}` here.
- `components/require-auth.tsx` wraps protected pages, reading `useAuth()` — UX-only; the real enforcement is server-side `requireAuth` middleware.

6 If you add a new resource, follow this checklist

Keep doing

- ✓ One hook per operation, named for what it does (`useCreateX`, not `useX.create`)
- ✓ Types defined inline in the hook that uses them
- ✓ Manual `isLoading/isError/error` state, explicit and readable
- ✓ Call through the shared `api` instance, never raw `axios/fetch`

Don't reach for

- ✗ Wiring in `fetch-client.ts` or `simple-cache.ts` — reference-only, not to be imported
- ✗ A new `services/` or `central types/` folder — DRYing this up is an explicit non-goal here
- ✗ React Query — would solve a caching problem this demo doesn't have

See also: `AXIOS-MIGRATION-EXPLAINED.pdf` (every real call site), `AXIOS-INTERCEPTORS-EXPLAINED.pdf` (the refresh engine), and `CLAUDE.md` for the full rationale behind these choices.